One Campus LAN: Integrating ARP, DHCP and Telnet with DHCP Snooping, ARP Poisoning, and Dynamic ARP Inspection

for Computer Networks Laboratory

Group members

Hrishikesh Bhanja	ARP module
Aritra Dutta	DHCP module
Subham Bhat	Telnet module and the integration

Class: BCSE III

Group: A1

College: Jadavpur University

Contents

1 Introduction **3**

1.1 What Was Built 3

2 The Campus Network **4**

3 Integration Architecture **5**

4 Implementation of the Links **5**

4.1 DHCP → ARP: Leases Become Snooping Bindings 5

4.2 ARP → DHCP: Conflict Detection and DHCPDECLINE 6

4.3 Telnet → ARP: Telnet Bytes as Frames on the Switch 6

4.4 Telnet → DHCP and ARP: Router Commands 6

4.5 The Replay 7

5 The Story and Its Results **8**

5.1 Running and Verification 12

6 Conclusion **13**

AI Acknowledgement **13**

1 Introduction

Our project basket is **ARP**, **DHCP** and **Telnet**. They are usually taught as three separate protocols at three different layers, yet a host joining a LAN needs all three, in order:

1. an **IP address**, from DHCP (RFC 2131);
2. the **MAC address of its next hop**, from ARP (RFC 826), after checking with ARP that its new address is not already in use (RFC 5227);
3. a **management session** to the router, which on many campus networks is still Telnet (RFC 854).

The three are also linked by security. ARP is unauthenticated, so an attacker can poison caches and redirect traffic. The most valuable traffic to steal is a Telnet login, which is clear text. The standard defence, **Dynamic ARP Inspection** (DAI), needs a trusted table of which MAC owns which IP, and that table is built by **DHCP snooping**. Finally, the administrator switches DAI on through the router's Telnet CLI.

The three modules were built separately by the members of Group A1: **ARP** by Hrishikesh Bhanja, **DHCP** by Aritra Dutta, and **Telnet** by Subham Bhat. This report describes the **integration**, written by Subham Bhat, in `MainProject`: one campus LAN, 192.168.1.0/24, on which the three modules run together and depend on one another, with a ten-step story that shows each dependency for real.

1.1 What Was Built

1. **An integration bus** (`campus.Bus`) that starts the DHCP cluster and the ARP lab's switch with all hosts, and joins them only through the modules' public APIs.
2. **DHCP → ARP**: every active lease becomes a DHCP-snooping binding on the switch; a declined lease removes it again.
3. **ARP → DHCP**: an address conflict found by an ARP probe makes the client send a real DHCPDECLINE.
4. **Telnet → ARP**: a relay carries every Telnet chunk as an Ethernet frame across the ARP lab's switch to the MAC address in the sender's ARP cache, so ARP poisoning really decides who receives the Telnet bytes.
5. **Telnet → DHCP and ARP**: router CLI commands that show and change the other modules' tables and enable DAI.
6. **A Packet Tracer–style dashboard** on `http://localhost:8089` that replays every packet each step produced, with a PDU panel and an event list.
7. **18 end-to-end checks** (`run.bat -verify`).

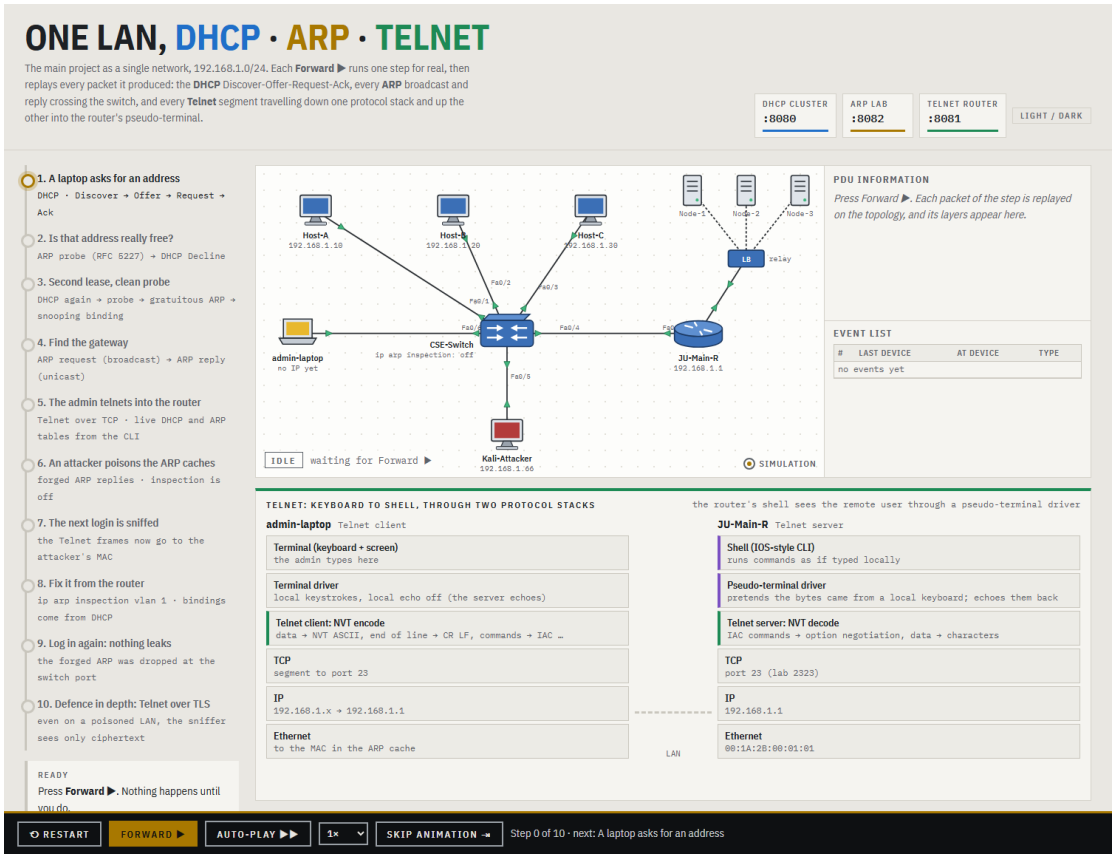


Figure 1: The campus LAN at the start of the story.

2 The Campus Network

Table 1: Stations on switch CSE-Switch

Station	Port	IP	MAC	Role
Host-A	Fa0/1	192.168.1.10 (static)	`AA:AA:AA:11:11:11`	Causes the address conflict
Host-B	Fa0/2	192.168.1.20 (static)	`BB:BB:BB:22:22:22`	
Host-C	Fa0/3	192.168.1.30 (static)	`CC:CC:CC:33:33:33`	
JU-Main-R	Fa0/4 (trusted)	192.168.1.1	`00:1A:2B:00:01:01`	Gateway, DHCP relay, Telnet VTY lines
Kali-Attacker	Fa0/5	192.168.1.66	`66:66:66:66:66:66`	ARP poisoning (man in the middle)
admin-laptop	Fa0/6	from DHCP	`3C:52:82:AD:00:01`	The newcomer

The DHCP cluster (a load balancer on UDP 6700 with consistent hashing, and three nodes on 6701–6703) serves the pool 192.168.1.10–150 with gateway 192.168.1.1. The Telnet server listens on 2323 (TLS 9923); the relay that carries its bytes across the simulated LAN listens on 2325 (TLS 9925). The three modules' own dashboards stay available on 8080 (DHCP), 8081 (Telnet) and 8082 (ARP).

3 Integration Architecture

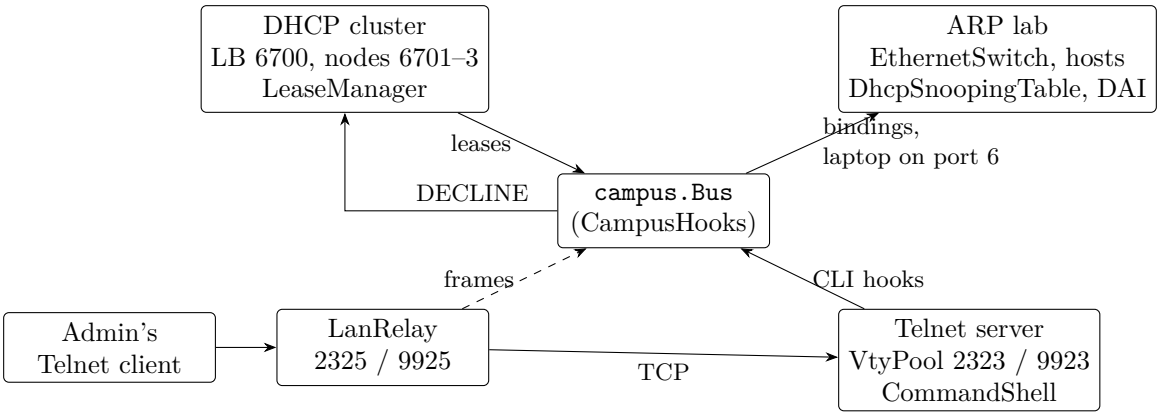


Figure 2: How the three modules are joined.

`run.bat` compiles the three modules from their own folders together with the integration package:

```
javac --release 11 -encoding UTF-8 -d out ^
      -sourcepath "..\ARP\src;..\DHCP\src;..\Telnet\src;src" ^
      src\campus\*.java
```

Table 2: Integration classes in `MainProject/src/campus`

Class	Job
`Bus`	Starts the DHCP cluster and the switch with all hosts; syncs leases into the snooping table; runs the probe, decline and poisoning actions; implements `CampusHooks`.
`LanRelay`	Passes the real Telnet TCP session through and sends each chunk as an Ethernet frame across the switch.
`Recorder`	Captures what each step produced: DHCP log lines, frames seen at the switch, Telnet bytes decoded into NVT tokens.
`Story`	The ten steps; each **Forward** performs real protocol actions and returns the replay.
`Dashboard`	HTTP API (`/api/state`, `/api/forward`, `/api/restart`) and the page.
`Verify`	The end-to-end checks.

Only two changes were made outside `MainProject`: the Telnet module gained the `CampusHooks` interface and the router commands that use it, and a compile error in the ARP module's `Hub.java` was fixed.

4 Implementation of the Links

4.1 DHCP → ARP: Leases Become Snooping Bindings

The bus reads every DHCP node's `LeaseManager`. Each ACTIVE lease is added to the switch's `DhcpSnoopingTable`. When a lease is DECLINED, its binding is removed and any static binding it had displaced (Host-A's, in the story) is restored.

```
1 public synchronized void syncLeases() {
2     for (DhcpServerNode n : nodes) for (LeaseRecord r : n.getLeaseManager().getAllLeases())
      {
```

```
        String key = r.getIpAddress() + "|" + r.getMacAddress().toUpperCase();
        if (r.getState() == LeaseRecord.State.DECLINED && bound.removeIf(k -> k.startsWith(r
    .getIpAddress() + "|"))) {
            DhcpSnoopingTable.Binding old = displaced.remove(r.getIpAddress());
            if (old != null) snooping.addBinding(old.ip, old.mac, old.port, old.hostname);
     // give .10 back to Host-A
            continue;
        }
        if (r.getState() != LeaseRecord.State.ACTIVE) continue;
        if (!bound.add(key)) continue;                                    // already bound
        int port = r.getMacAddress().equalsIgnoreCase(ADMIN_MAC) ? ADMIN_PORT : 0;
        IpAddress lip = IpAddress.parse(r.getIpAddress());
        DhcpSnoopingTable.Binding prev = snooping.getBinding(lip);
        if (prev != null && !prev.mac.equals(MacAddress.parse(r.getMacAddress()))) displaced
    .put(r.getIpAddress(), prev);
        snooping.addBinding(lip, MacAddress.parse(r.getMacAddress()), port, r.getHostname())
    ;
    }
}
```

Listing 1: `Bus.syncLeases()`.

4.2 ARP → DHCP: Conflict Detection and DHCPDECLINE

After DORA the laptop probes its new address with an ARP request whose sender IP is 0.0.0.0 (RFC 5227). Host-A owns .10 and defends it with a gratuitous ARP. The bus detects the defence and calls the DHCP client's `sendDecline`, which sends a real DHCPDECLINE over UDP; the cluster marks the address as declined and the second DORA yields .11.

4.3 Telnet → ARP: Telnet Bytes as Frames on the Switch

`LanRelay` forwards the real TCP session between the admin's client and the Telnet server. For every chunk it also asks the bus to send an IPv4 frame from the sender's network interface to the MAC address that the sender's ARP cache holds for the destination IP. When the cache is clean, the frame reaches the router's port; when it is poisoned, it reaches the attacker's port, and the attacker's frame handler logs the Telnet text.

```
public void telnetBytes(boolean toRouter, byte[] b, int n, boolean tls) {
    String text = tls ? "TLS record, " + n + " bytes (encrypted)" : printable(b, n);
    String payload = "TELNET " + (toRouter ? ip + " > " + GATEWAY + ":23" : GATEWAY + ":23 >
     " + ip) + " | " + text;
    HostNode from = toRouter ? admin : router;
    IpAddress dst = toRouter ? router.getNic().getIp() : IpAddress.parse(ip);
    MacAddress mac = from.getArpTable().lookupMac(dst);                 // ask the sender's
     ARP cache
    if (mac == null) { from.sendArpRequest(dst); mac = from.getArpTable().lookupMac(dst); }
    if (mac == null) return;
    int toPort = mac.equals(attacker.getNic().getMac()) ? 5 : toRouter ? 4 : ADMIN_PORT;
    rec.telnet(toRouter, b, n, tls, toPort, ip);                        // for the replay
    from.getNic().send(new EthernetFrame(mac, from.getNic().getMac(),
            EthernetFrame.ETHERTYPE_IPV4, payload.getBytes(StandardCharsets.UTF_8)));
}
```

Listing 2: `Bus.telnetBytes()`: who receives the Telnet chunk is decided by ARP.

4.4 Telnet → DHCP and ARP: Router Commands

The Telnet server's `CommandShell` calls the bus through `CampusHooks`:

- `show ip dhcp binding`: the live leases from the DHCP cluster;
- `show ip arp`, `show mac address-table`: the router's ARP cache and the switch's CAM table;
- `show ip arp inspection`, `clear arp-cache`;
- `ip arp inspection vlan 1` and `no ip arp inspection vlan 1` in configuration mode: switch DAI on or off.

4.5 The Replay

After each step, `Recorder` returns the packets it produced. The dashboard animates them on a Packet Tracer–style topology:

- a DHCP DISCOVER or REQUEST fans out to every port (PCs drop it with `X`; the router relays it to the load balancer, which hashes the MAC to one node);
- an ARP request floods every port (`OK` at the owner) and the reply returns as unicast;
- a Telnet segment walks down the laptop's stack (terminal, terminal driver, Telnet client and NVT, TCP, IP, Ethernet) and up the router's to the **pseudo-terminal driver** and the shell, showing the real NVT bytes, for example `IAC DO ECHO` = `FF FD 01`.

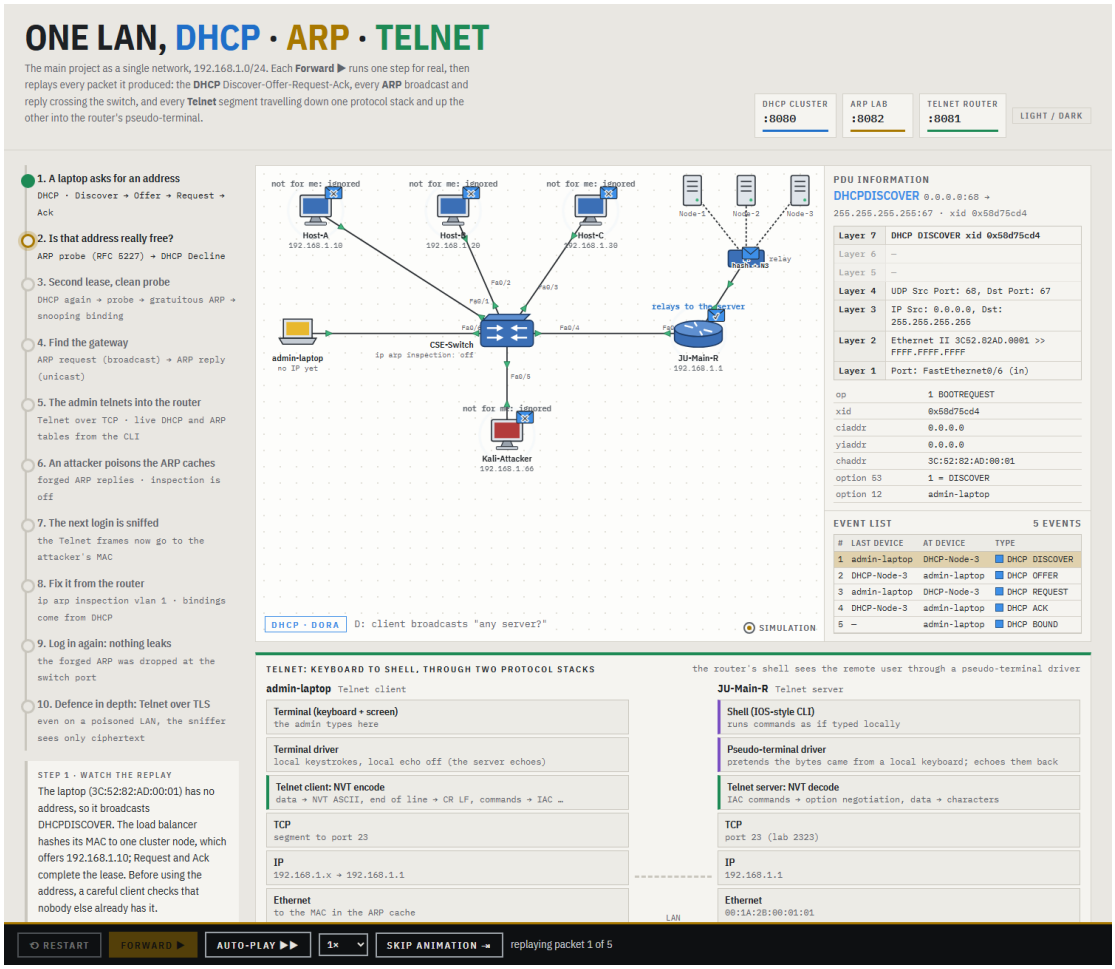


Figure 3: DHCP DISCOVER as a layer-2 broadcast, relayed to the cluster.

5 The Story and Its Results

Each press of **Forward** runs one step for real. The outcomes below are those produced by the module.

Table 3: The ten steps

#	Step	What happens
1	A laptop asks for an address	Real DORA over UDP; the cluster offers 192.168.1.10.
2	Is that address really free?	ARP probe (sender IP 0.0.0.0); Host-A defends .10 with a gratuitous ARP; the laptop sends DHCPDECLINE. Host-A's snooping binding is restored.
3	Second lease, clean probe	DORA again gives .11; the probe gets no answer; gratuitous ARP; the binding .11 ↔ laptop on port 6 appears in the snooping table.
4	Find the gateway	ARP request for 192.168.1.1 floods the switch; the router replies by unicast; the laptop's cache and the switch's CAM table fill in.
5	The admin telnets into the router	Login as `admin`; `show ip dhcp binding` and `show ip arp` print the live tables of the other two modules.
6	An attacker poisons the ARP caches	Forged ARP replies (inspection is off): the laptop now maps 192.168.1.1 to the attacker's MAC; the path is drawn in red.
7	The next login is sniffed	The Telnet frames go to port 5; the attacker's notebook shows `admin` / `cisco`.
8	Fix it from the router	`ip arp inspection vlan 1` typed in the same Telnet session switches DAI on.
9	Log in again: nothing leaks	The forged replies are dropped at the switch port because no DHCP binding supports them; the login reaches the router; the notebook stays empty.
10	Defence in depth: Telnet over TLS	With inspection off and the LAN poisoned again, the session runs over TLS; the attacker sees only encrypted records.

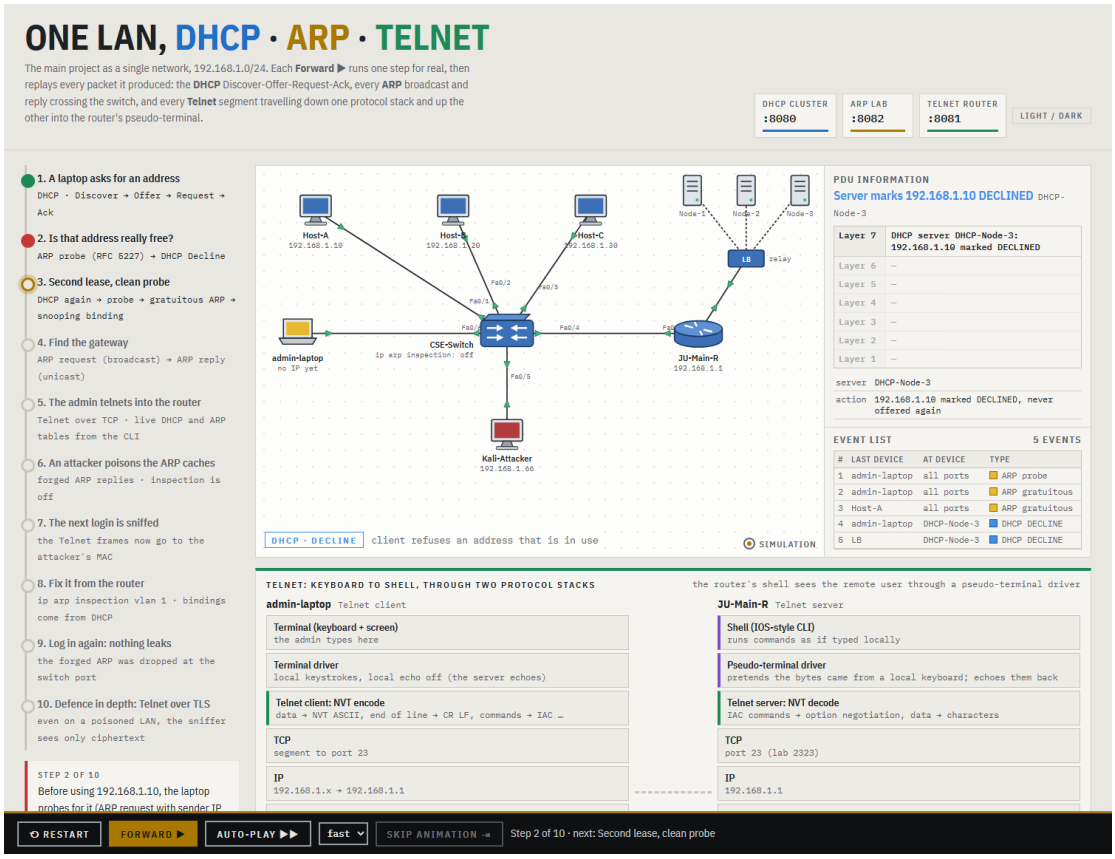


Figure 4: Step 2: the ARP probe finds Host-A on .10 and the laptop declines the lease.

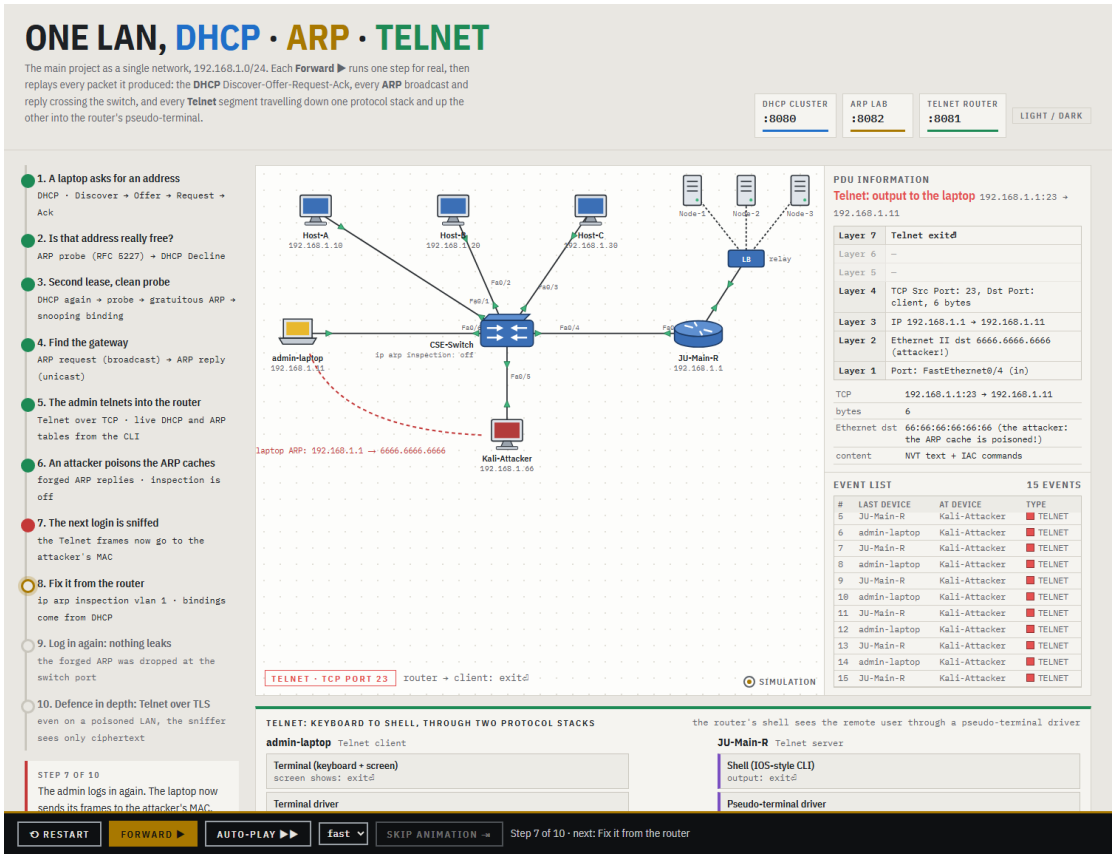


Figure 5: Step 7: after ARP poisoning, the attacker reads the Telnet credentials.

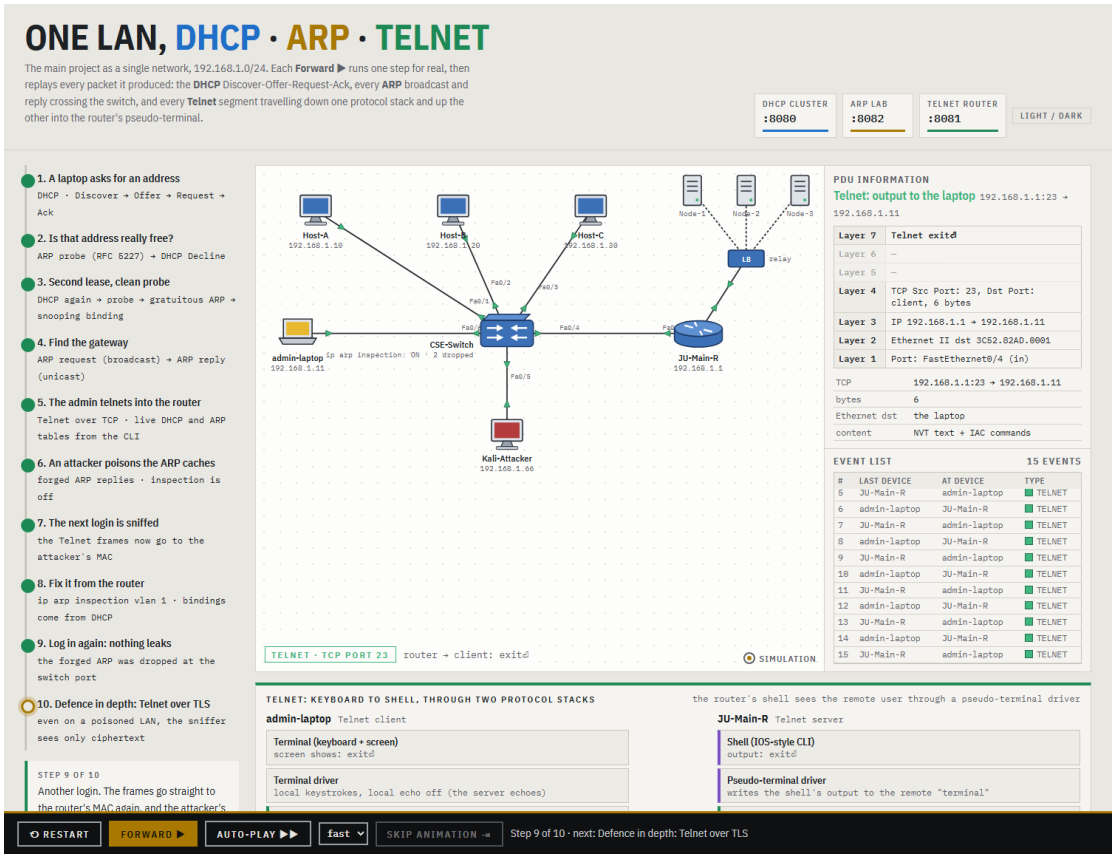


Figure 6: Step 9: with DAI on, the forged ARP replies are dropped using the DHCP bindings.

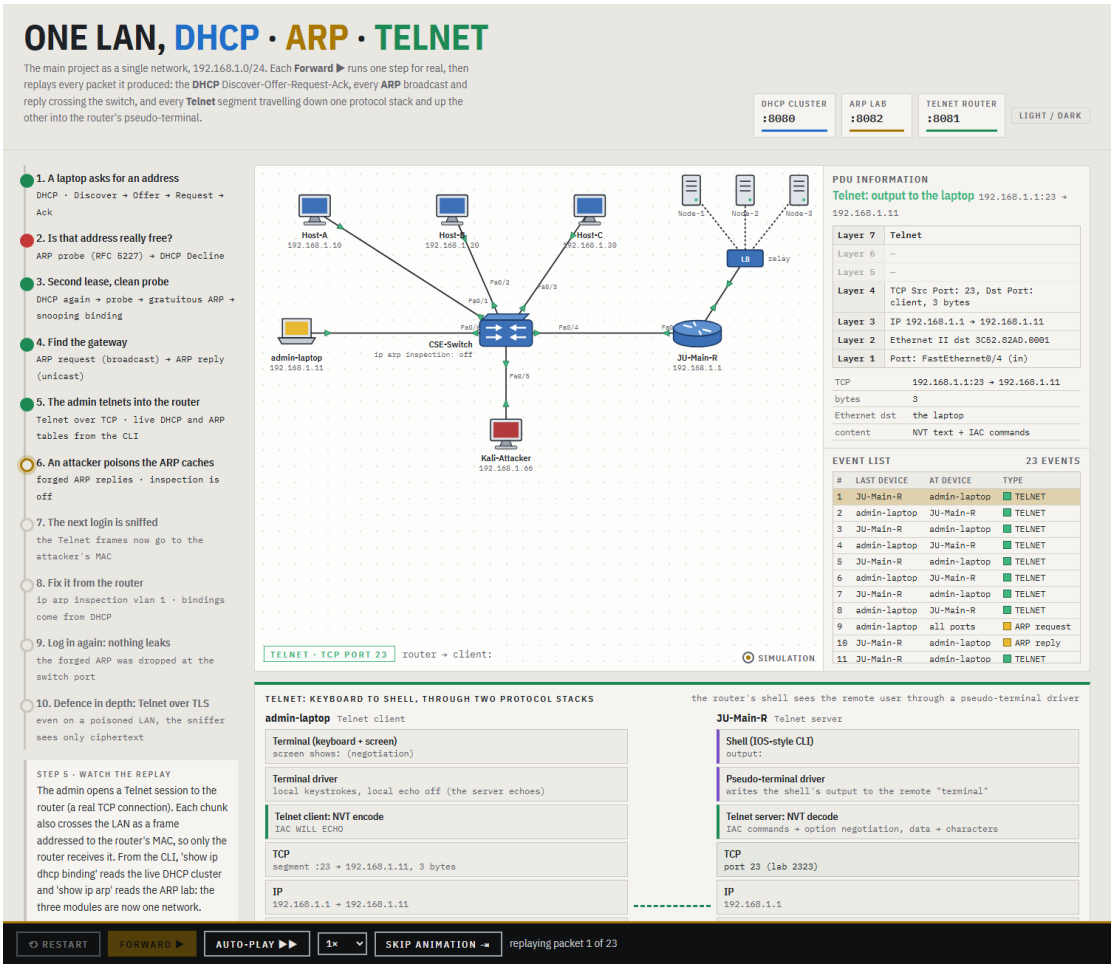


Figure 7: A Telnet segment walking up the router's stack to the pseudo-terminal driver.

5.1 Running and Verification

```
cd MainProject
run.bat                    # compiles ARP + DHCP + Telnet + campus; story at http://localhost
    :8089
run.bat --verify           # 18 end-to-end checks
telnet 127.0.0.1 2325      # a Telnet session across the simulated LAN (admin / cisco)
```

Listing 3: Running the integrated project.

Table 4: End-to-end checks (all passed)

Real DORA over UDP: the cluster leases 192.168.1.10 to the laptop
ARP → DHCP: the probe finds Host-A on .10 and the client sends DHCPDECLINE
After the decline, .10's snooping binding is Host-A's again; the second lease is a different, conflict-free address
DHCP → ARP: the lease is in the switch's DHCP-snooping table
The laptop's ARP cache maps 192.168.1.1 to the router's MAC; the switch learned the laptop's MAC on port 6
Telnet → DHCP: `show ip dhcp binding` lists the live lease; Telnet → ARP: `show ip arp` lists the laptop
No interception before the attack; ARP poisoning succeeds while inspection is off
ARP → Telnet: the attacker reads the Telnet user name and password
Telnet → ARP: `ip arp inspection vlan 1` typed on the router enables DAI; forged replies are dropped using the DHCP bindings
The next Telnet login is not intercepted; on a poisoned LAN a TLS session leaks only encrypted records
After Restart the story begins again from an empty lease database

6 Conclusion

1. **The three protocols depend on each other.** DHCP supplies the address and the trusted bindings, ARP checks the address and resolves the gateway, and Telnet manages the router. Integrating the modules made each dependency visible as a real message: a DHCPDECLINE caused by an ARP probe, and a snooping binding created by a DHCP ACK.

2. **A weakness in one layer breaks another.** Because ARP accepts unsolicited replies, poisoning redirected real Telnet bytes to the attacker, who read the clear-text password.

3. **The fix comes from a third protocol.** Dynamic ARP Inspection, using the bindings that DHCP snooping built and enabled from the Telnet CLI, dropped the forged replies; Telnet over TLS protected the login even when inspection was off. Layered defences are needed because each one alone can be misconfigured.

AI Acknowledgement

Artificial Intelligence tools were used as an aid during this project. Specifically, AI assistance was used for:

- **Protocol cross-checking:** comparing message formats, reply codes, state machines and byte offsets against the RFCs cited in this report.
- **Code review and testing:** suggesting edge cases for the automated verification suite and reviewing concurrency and error handling.
- **Writing and visualization:** drafting, structuring and refining the explanations, tables, diagrams and LaTeX source of this report.

All protocol implementations, Java source code, experiments, measurements and final conclusions were run, reviewed and verified by the author, who takes full responsibility for the content of this report.